

National University of Computer and Emerging Sciences



AL-2002: Artificial Intelligence Lab Lab Manual 11

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives	3
1 Introduction to Hugging Face	3
1.1 What is a Hugging Face?	3
1.2 Hugging Face Ecosystem	4
1.3 Hugging Face Hub	5
1.4 Hugging Face Transformers Library	5
2 Using Pretrained Models with Pipeline API	6
2.1 What is a Pipeline?	6
2.2 Sentiment Analysis Example	6
2.3 Text Generation Example	7
3 Loading Models and Tokenizers	8
3.1 AutoTokenizer	8
3.2 AutoModel	9
3.3 Model heads: Making sense out of numbers	10
3.4 Postprocessing the output	11

Objectives

After performing this lab, students shall be able to:

- ✓ Understand the Hugging Face ecosystem
- ✓ Use pretrained transformer models
- ✓ Load models using AutoTokenizer and AutoModel
- ✓ Perform inference using pretrained NLP models
- ✓ Interpret outputs generated by transformer models

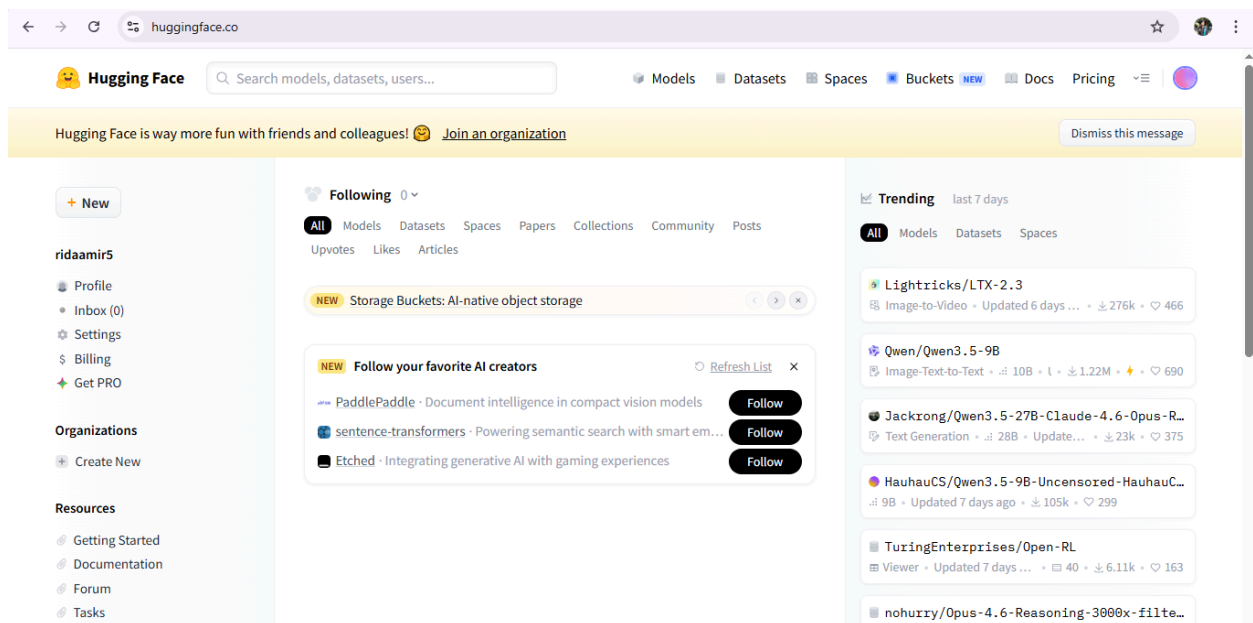
1 Introduction to Hugging Face

Modern Natural Language Processing (NLP) applications rely heavily on pretrained transformer models. Hugging Face provides an ecosystem that makes it easy to access, train and deploy these models.

The Hugging Face platform provides tools and libraries that simplify the process of building state-of-the-art AI applications in areas such as:

- Natural Language Processing
- Computer Vision
- Speech Recognition

The platform hosts thousands of pretrained models that can be used with only a few lines of code.



1.1 What is Hugging Face?

Hugging Face is an open-source AI company that provides tools for building machine learning

applications using transformer-based architectures.

It provides:

- Pretrained models
- Datasets
- APIs
- Model sharing platform

These resources allow researchers and developers to quickly build applications such as:

- Chatbots
- Text summarization systems
- Question answering systems
- Language translation tools

1.2 Hugging Face Ecosystem

The Hugging Face ecosystem consists of several major components.

Models

Pretrained models for tasks such as:

- Text classification
- Translation
- Question answering
- Text generation
- Speech recognition

Examples of popular models include:

- BERT
- GPT
- RoBERTa
- T5
- DistilBERT

Datasets

The datasets library provides access to thousands of machine learning datasets for tasks such as NLP, computer vision, and speech processing.

It allows easy downloading, preprocessing, and loading of datasets.

Hugging Face Hub

The Hugging Face Hub is an online platform where developers can:

- Share models
- Upload datasets
- Collaborate with the community
- Download pretrained models

The hub hosts hundreds of thousands of models ready for immediate use.

1.3 Hugging Face Hub

The Hugging Face Hub acts as a repository similar to GitHub but designed specifically for machine learning models.

Features include:

- Model versioning
- Model sharing
- Documentation and examples
- Community contributions

Users can easily download pretrained models directly from the hub using Python libraries.

1.4 Hugging Face Transformers Library

The Transformers library is the core library of Hugging Face used for working with transformer-based models.

It provides implementations of popular architectures such as:

- BERT
- GPT
- RoBERTa
- T5
- XLNet

Key features include:

- Pretrained models
- Easy-to-use APIs
- Integration with PyTorch and TensorFlow
- Support for training and inference

```
pip install transformers
```

2 Using Pretrained Models with Pipeline API

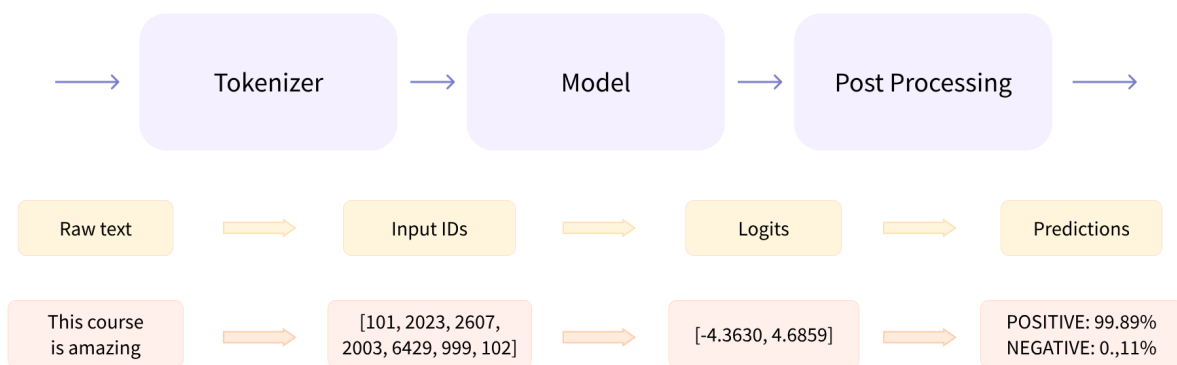
The easiest way to use Hugging Face models is through the **pipeline API**. A pipeline allows users to perform complex NLP tasks with minimal code.

2.1 What is a Pipeline?

A pipeline automatically handles several steps:

- Loading the pretrained model
- Loading the tokenizer
- Preprocessing input text
- Running the model
- Returning readable output

The pipeline API simplifies inference to just a few lines of code.



2.2 Sentiment Analysis Example

Sentiment analysis determines whether a piece of text expresses **positive or negative sentiment**.

First import the pipeline module.

```
from transformers import pipeline
```

Create a sentiment analysis pipeline.

```
sentiment_pipeline = pipeline("sentiment-analysis")
```

Run the model on some text.

```
result = sentiment_pipeline("I love learning Artificial Intelligence!")
print(result)
```

```
[{'label': 'POSITIVE', 'score': 0.9996731281280518}]
```

label indicates the predicted class, score represents the confidence of the model.

2.3 Text Generation Example

Text generation models can generate human-like text.

```
generator = pipeline("text-generation", model="gpt2")
output = generator("Artificial Intelligence will", max_new_tokens=30)
print(output)
```

```
{'generated_text': 'Artificial Intelligence will be a fundamental
component of any future AI, whether it be a computer or a machine.\n\nI
think that the most important thing to remember'}}
```

```
generator = pipeline("text-generation", model="gpt2")
output = generator("Artificial Intelligence will", max_new_tokens=30,
num_return_sequences=5)
print(output)
```

```
{'generated_text': "Artificial Intelligence will be able to understand
what you are saying, and do it by analyzing your mental state.\n\nIf
you're a computer scientist, the machine can"},
```

```
{'generated_text': 'Artificial Intelligence will be required to be able to
understand and respond to a variety of human-level topics, including
human-related activities, language, social interactions, and'},
```

```
{'generated_text': "Artificial Intelligence will likely be a big part of
the AI revolution, and we're already seeing it being implemented in
artificial intelligence.\n\nThis article will discuss some of"},
```

```
{'generated_text': "Artificial Intelligence will be used to search for new
things, like better AI, and if they're working properly you can expect to
see them used to solve your problems."},
```

```
{'generated_text': 'Artificial Intelligence will be available in the
future.\n\nThe research was conducted by University of California,
Berkeley Artificial Intelligence Laboratory (AI Lab) and U.S.'}]
```

The model continues the input text based on patterns learned during training.

Other pipeline functions: <https://huggingface.co/learn/llm-course/en/chapter1/3>

3 Loading Models and Tokenizers

Sometimes we want more control over the model rather than using pipelines. For this purpose, Hugging Face provides **Auto classes**.

These classes automatically load the correct model architecture.

3.1 AutoTokenizer

Like other neural networks, Transformer models can't process raw text directly, so the first step of our pipeline is to convert the text inputs into numbers that the model can make sense of. To do this we use a tokenizer, which will be responsible for:

- Splitting the input into words, subwords, or symbols (like punctuation) that are called tokens
- Mapping each token to an integer
- Adding additional inputs that may be useful to the model

All this preprocessing needs to be done in exactly the same way as when the model was pretrained, so we first need to download that information from the Model Hub. To do this, we use the `AutoTokenizer` class and its `from_pretrained()` method. Using the checkpoint name of our model, it will automatically fetch the data associated with the model's tokenizer and cache it (so it's only downloaded the first time you run the code below).

Consider the following example

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

Once we have the tokenizer, we can directly pass our sentences to it and we'll get back a dictionary that's ready to feed to our model! The only thing left to do is to convert the list of input IDs to tensors since **transformer models only accept tensors as input**.

```
raw_inputs = [
    "I've been waiting for a HuggingFace course my whole life.",
```

```

    "I hate this so much!",
]
inputs = tokenizer(raw_inputs, padding=True, truncation=True,
return_tensors="pt")
print(inputs)

```

`raw_inputs`: Your text input(s).

`padding=True`: Pads shorter sequences so all inputs have the same length.

`truncation=True`: Cuts text if it exceeds the model's maximum length.

`return_tensors="pt"`: Converts the tokenized output into PyTorch tensors.

```

{
    'input_ids': tensor([
        [ 101,  1045,  1005,  2310,  2042,  3403,  2005,  1037, 17662,
12172, 2607,  2026,  2878,  2166,  1012,   102],
        [ 101,  1045,  5223,  2023,  2061,  2172,   999,   102,    0,
0,      0,      0,      0,      0,      0,      0]
    ]),
    'attention_mask': tensor([
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]
    ])
}

```

3.2 AutoModel

The `AutoModel` class loads the pretrained neural network architecture.

```

from transformers import AutoModel
model = AutoModel.from_pretrained("distilbert-base-uncased")

```

When loading the model, the library automatically downloads the pretrained weights from the Hugging Face Hub. This architecture contains only the base Transformer module: given some inputs, it outputs what we'll call hidden states, also known as features. For each model input, we'll retrieve a high-dimensional vector representing the contextual understanding of that input

by the Transformer model. While these hidden states can be useful on their own, they're usually inputs to another part of the model, known as the **head**. Each nlp task can be performed with the same model with just using a different head.

The vector output by the Transformer module is usually large. It generally has three dimensions:

- **Batch size:** The number of sequences processed at a time (2 in our example).
- **Sequence length:** The length of the numerical representation of the sequence (16 in our example).
- **Hidden size:** The vector dimension of each model input.

It is said to be “high dimensional” because of the last value. The hidden size can be very large (768 is common for smaller models, and in larger models this can reach 3072 or more).

We can see this if we feed the inputs we preprocessed to our model:

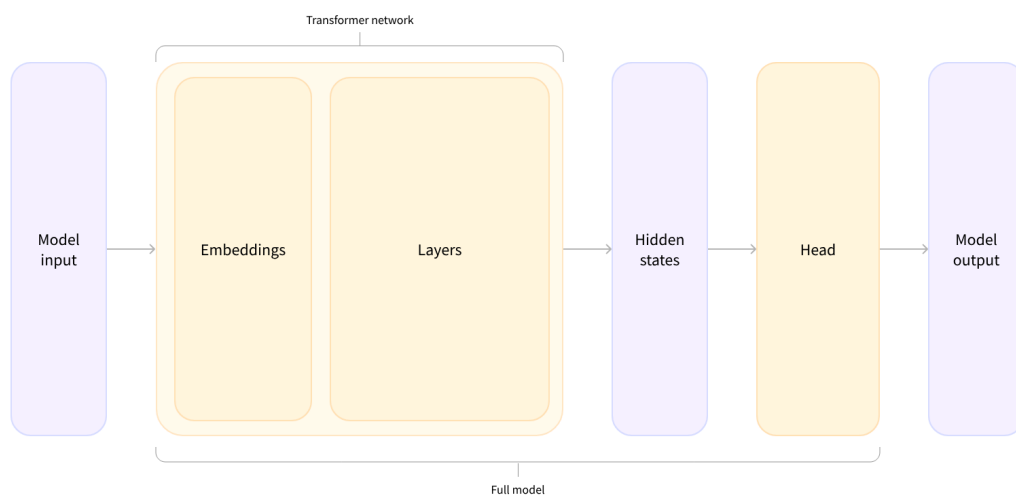
```
outputs = model(**inputs)
print(outputs.last_hidden_state.shape)
```

```
torch.Size([2, 16, 768])
```

The output here shows number of sentences=2, tokens in each sentence=16 and embedding vector size for each token=768

3.3 Model heads: Making sense out of numbers

The model heads take the high-dimensional vector of hidden states as input and project them onto a different dimension. They are usually composed of one or a few linear layers:



The output of the Transformer model is sent directly to the model head to be processed.

In this diagram, the model is represented by its embeddings layer and the subsequent layers. The embeddings layer converts each input ID in the tokenized input into a vector that represents the associated token. The subsequent layers manipulate those vectors using the attention mechanism to produce the final representation of the sentences.

There are many different architectures available in Transformers, with each one designed around tackling a specific task. eg:

- *Model (retrieve the hidden states)
- *ForCausalLM
- *ForMaskedLM
- *ForMultipleChoice
- *ForQuestionAnswering
- *ForSequenceClassification
- *ForTokenClassification

For our example, we will need a model with a sequence classification head (to be able to classify the sentences as positive or negative). So, we won't actually use the AutoModel class, but AutoModelForSequenceClassification. (AutoModel loads only the base transformer (like the encoder of DistilBERT or BERT) which requires a head to build your own classifier on top)

```
from transformers import AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)
outputs = model(**inputs)
```

Now if we look at the shape of our outputs, the dimensionality will be much lower: the model head takes as input the high-dimensional vectors we saw before, and outputs vectors containing two values (one per label):

```
print(outputs.logits.shape)
```

```
torch.Size([2, 2])
```

Since we have just two sentences and two labels, the result we get from our model is of shape 2 x 2

3.4 Postprocessing the output

The values we get as output from our model don't necessarily make sense by themselves. Let's take a look:

```
print(outputs.logits)
```

```
tensor([[ -1.5607,  1.6123],
        [ 4.1692, -3.3464]], grad_fn=<AddmmBackward0>)
```

Our model predicted [-1.5607, 1.6123] for the first sentence and [4.1692, -3.3464] for the second one. Those are not probabilities but logits, the raw, unnormalized scores outputted by the last layer of the model. To be converted to probabilities, they need to go through a SoftMax layer (all Transformers models output the logits, as the loss function for training will generally fuse the last activation function, such as SoftMax, with the actual loss function, such as cross entropy):

```
import torch

predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)
print(predictions)
```

```
tensor([[4.0195e-02, 9.5980e-01],
        [9.9946e-01, 5.4418e-04]], grad_fn=<SoftmaxBackward0>)
```

Now we can see that the model predicted [0.0402, 0.9598] for the first sentence and [0.9995, 0.0005] for the second one. These are recognizable probability scores.

To get the labels corresponding to each position, we can inspect the id2label attribute of the model config:

```
model.config.id2label
```

```
{0: 'NEGATIVE', 1: 'POSITIVE'}
```

Now we can conclude that the model predicted the following:

- First sentence: NEGATIVE: 0.0402, POSITIVE: 0.9598
- Second sentence: NEGATIVE: 0.9995, POSITIVE: 0.0005

For more information: <https://huggingface.co/learn/llm-course/en/chapter2/6>